

Ligador, Bibliotecas e Ligação Estática

Seminário de Software Básico

Alberto Henrique

Caio Veloso

Gabriel Ramos

Ícaro Vinícius

Vinícius Macedo

Grupo 2

2026

Contexto

Definições

Processos e Etapas

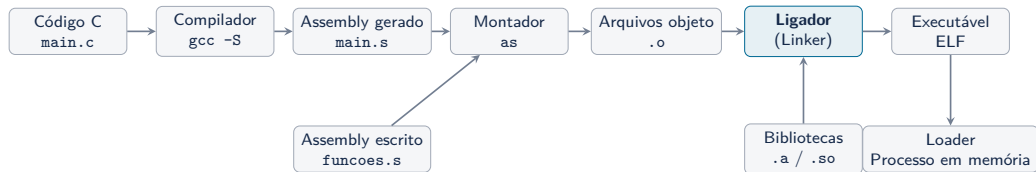
Vantagens e Desvantagens

Exemplo Prático

Considerações Finais

Contexto

Cada Ferramenta Produz a Entrada da Próxima



As fronteiras importam

O gcc é um driver: com -S, ele para antes do montador. O **linker constrói** o executável; o **loader** mapeia seus segmentos na memória e inicia o processo.

Definições

O que é o Ligador (Linker)?

Definição

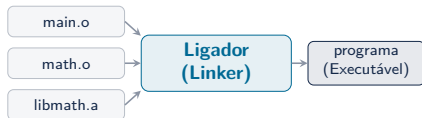
Programa que **combina** múltiplos arquivos-objeto e bibliotecas em um **único executável**.

Entrada: arquivos .o + bibliotecas

Processo: resolução de símbolos +
relocação

Saída: executável binário

Exemplos: `ld` (GNU), `link.exe` (MSVC)



Definição

Coleções de **código pré-compilado** empacotadas para **reutilização**.

Biblioteca Estática

- Extensão: `.a` / `.lib`
- Módulos **necessários** incorporados ao executável
- Criada com `ar rcs`

Biblioteca Dinâmica

- Extensão: `.so` / `.dll`
- Carregada em **runtime**
- **Compartilhada** entre programas

Definição

Processo realizado em **tempo de ligação** no qual os **módulos necessários** de uma biblioteca estática são incorporados ao executável.

- A biblioteca .a não precisa acompanhar o programa.
- O linker seleciona os objetos que resolvem símbolos pendentes.
- O resultado tende a ser mais independente, porém maior.

Atenção: usar uma biblioteca .a não torna todo o executável estático; outras bibliotecas podem permanecer dinâmicas.

Ligação Estática

Código do Programa

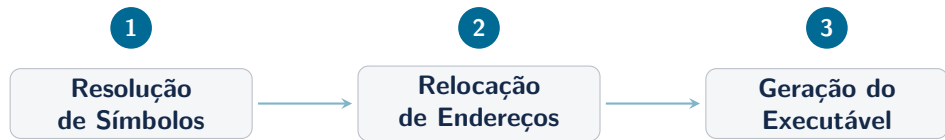
Objetos necessários da .a

Ligação Dinâmica

Código do Programa

Referência a Biblioteca .so

Processos e Etapas



1. **Resolução de Símbolos:** associa cada referência à sua definição
2. **Relocação:** organiza seções e corrige referências dependentes da posição
3. **Geração:** produz o ELF final, seus segmentos e o ponto de entrada

Como funciona?

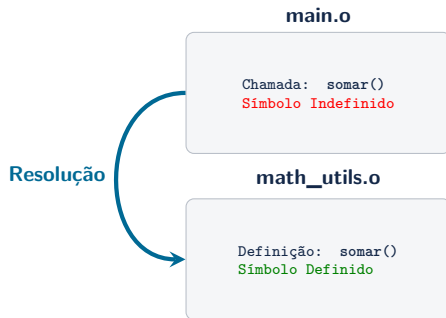
- Cada .o contém uma **tabela de símbolos**.
- Um **global definido** pode resolver referências externas; um símbolo local só vale no próprio módulo.
- Um **indefinido** ainda precisa de uma definição externa.

Quando a resolução falha

undefined reference: nenhuma definição encontrada.

multiple definition: mais de uma definição global forte.

Uma definição fraca pode ser substituída por uma forte.



Problema

Um arquivo `.o` é **relocável**: contém offsets relativos às suas seções, símbolos e registros que indicam quais posições deverão ser corrigidas pelo linker.

Antes (referências relocáveis)

```
main.o: call somar → destino  
pendente  
math.o: somar no offset 0x0000 de sua  
.text
```

Depois (layout final)

```
main.o: call com deslocamento  
corrigido  
math.o: função posicionada, por  
exemplo, em 0x401020
```

O linker **organiza as seções**, atribui posições finais e aplica as relocações. Em AMD64, muitas referências finais são relativas ao registrador de instruções (RIP).

Arquivo gerado

Formato **ELF** (Linux) ou **PE** (Windows):

- **ELF Header:** metadados e ponto de entrada
- **Program Headers:** segmentos que o loader mapeia na memória
- **.text** e **.rodata:** código e constantes
- **.data** e **.bss:** dados inicializados e não inicializados
- **Tabela de símbolos:** útil para análise, mas pode ser removida com `strip`

ELF Header

Program Headers

.text / **.rodata**

.data / **.bss**

Tabela de símbolos (opcional)

Vantagens e Desvantagens

Vantagens

- **Distribuição simplificada** em sistemas compatíveis
- Menor risco de biblioteca **ausente ou incompatível**
- **Versão controlada** da biblioteca incorporada
- Inicialização potencialmente mais simples, sem parte da resolução dinâmica

Desvantagens

- O executável **tende a ser maior** que um equivalente com bibliotecas compartilhadas
- Pode haver **duplicação** de código entre programas
- Atualizar código incorporado exige **religação e redistribuição**
- Algumas bibliotecas de sistema podem não oferecer versão estática

Casos comuns: sistemas embarcados e utilitários de linha de comando. Containers e binários Go/Rust dependem da configuração e das bibliotecas utilizadas.

Exemplo Prático

Código Fonte: C (Interface)

string_utils.h

```
1  #ifndef STRING_UTILS_H
2  #define STRING_UTILS_H
3
4  long long my_strlen(const char *
        str);
5  void my_reverse(char *str);
6
7  #endif
```

main.c

```
1  #include <stdio.h>
2  #include "string_utils.h"
3
4  int main() {
5      char texto[] = "Software
        Basico";
6      long long len = my_strlen(
        texto);
7      printf("Tamanho: %lld\n", len)
        ;
8      my_reverse(texto);
9      printf("Invertido: %s\n",
        texto);
10     return 0;
11 }
```

A ABI Conecta o Código C ao Assembly

my_strlen.s

```
1  .section .text
2  .globl my_strlen
3  .type my_strlen, @function
4  my_strlen:
5      xorq %rax, %rax
6  .loop:
7      cmpb $0, (%rdi,%rax)
8      je .end
9      incq %rax
10     jmp .loop
11 .end:
12     ret
```

ABI System V AMD64

- %rdi: primeiro argumento, o ponteiro para a string
- %rax: valor retornado por my_strlen
- %rax, %rcx, %rdx, %rsi e %rdi são caller-saved

my_reverse também recebe a string em %rdi, altera o buffer diretamente e não possui valor de retorno.

my_reverse Busca as Extremidades e Troca os Bytes

1. Localizar o fim

```
1  .section .text
2  .globl my_reverse
3  .type my_reverse, @function
4  my_reverse:
5      movq %rdi, %rsi
6  .find_end:
7      cmpb $0, (%rsi)
8      je .prepare
9      incq %rsi
10     jmp .find_end
11 .prepare:
12     cmpq %rdi, %rsi
13     je .done
14     decq %rsi
15     movq %rdi, %rdx
```

2. Trocar os bytes

```
1  .swap:
2      cmpq %rsi, %rdx
3      jae .done
4      movb (%rdx), %al
5      movb (%rsi), %cl
6      movb %cl, (%rdx)
7      movb %al, (%rsi)
8      incq %rdx
9      decq %rsi
10     jmp .swap
11 .done:
12     ret
```

O código usa registradores caller-saved e testa a string vazia antes de decq.

Processo de Ligação Estática

1. Montagem e criação da biblioteca .a

```
1 $ as my_strlen.s -o my_strlen.o
2 $ as my_reverse.s -o my_reverse.o
3 # nao_usada.s exporta funcao_nao_usada e executa apenas ret
4 $ as nao_usada.s -o nao_usada.o
5 $ ar rcs libstrutils.a my_strlen.o my_reverse.o nao_usada.o
```

2. Programa C e duas formas de ligação

```
1 $ gcc -c main.c -o main.o
2 # somente libstrutils.a ligada estaticamente; libc pode ser dinamica
3 $ gcc main.o ./libstrutils.a -o programa_misto
4 # executavel completamente estatico (requer libc estatica instalada)
5 $ gcc -static main.o ./libstrutils.a -o programa
```

A ordem importa: o objeto que cria as referências aparece antes da biblioteca que as resolve.

Antes da ligação

```
1 $ nm main.o | grep -E "my_strlen|my_reverse"
2           U my_reverse
3           U my_strlen
```

Execução

```
1 $ ./programa
2 Tamanho: 15
3 Invertido: ocisaB erawtfoS
```

Tipo do arquivo

```
1 $ file programa
2 ELF 64-bit LSB executable, x86-64,
3 statically linked
```

Depois da ligação - endereços variam

```
1 $ nm programa | grep -E "my_strlen|my_reverse"
2 00000000004010a0 T my_strlen
3 00000000004010c0 T my_reverse
```

Módulo não utilizado

```
1 $ nm programa | grep funcao_nao_usada
2 # nenhuma saída
```

Verificação com ldd

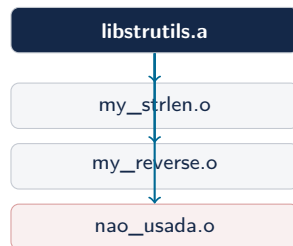
```
1 $ ldd programa
2 not a dynamic executable
```

Conteúdo do arquivo

```
1 $ ar t libstrutils.a
2 my_strlen.o
3 my_reverse.o
4 nao_usada.o
```

Resumo dos símbolos exportados

```
1 $ nm -g --defined-only libstrutils.a
2 my_strlen.o:
3 0000000000000000 T my_strlen
4 my_reverse.o:
5 0000000000000000 T my_reverse
6 nao_usada.o:
7 0000000000000000 T funcao_nao_usada
```



Seleção por módulo

Como nenhum símbolo de `nao_usada.o` é referenciado, o linker não extrai esse módulo da biblioteca.

Considerações Finais

Ligador

Combina .o's
em executável

Biblioteca

Código pré-compilado
para reutilização

Lig. Estática

Incorpora módulos
necessários da .a

- O Linker é **essencial** no processo de construção do programa
- Ligação estática → distribuição mais simples em sistemas **compatíveis**
- Trade-off frequente: possível aumento de **tamanho** vs menor dependência em tempo de execução

- STALLINGS, William. *Arquitetura e Organização de Computadores*. 8ª ed. Pearson, 2010.
- Material da disciplina de Software Básico: representação de dados, Assembly, pilha, procedimentos e chamadas de sistema.
- BRYANT, R. E.; O'HALLARON, D. R. *Computer Systems: A Programmer's Perspective*. 3ª ed. Pearson, 2015. Cap. 7.
- LEVINE, J. R. *Linkers and Loaders*. Morgan Kaufmann, 1999.
- GNU Binutils: documentação de ld, ar, nm e readelf.
<https://www.gnu.org/software/binutils/>
- *System V Application Binary Interface: AMD64 Architecture Processor Supplement*.

Perguntas?

Obrigado pela atenção!